

Implementação do esquema totalmente homomórfico sobre inteiros de chave reduzida.

Guilherme Rodrigues Bilar, Luan Cardoso dos Santos, Fabio Dacêncio Pereira
COMPSI- Computing and Information Systems Research Lab – Centro Universitário
Eurípedes de Marília
Avenida Higino Muzi Filho, 529 - Cep 17525-901 Marília/SP
lcsantos.lcsantos@gmail.com; guibilar@gmail.com; prof.fabiopereira@gmail.com

Resumo: Neste artigo é implementado o esquema totalmente homomórfico de chave reduzida (DGHV sobre inteiros) proposto por Jean-Sébastien Coron, Avradip Mandal, David Naccache e Mehdi Tibouchi, que foi publicado na conferência CRYPTO 2011, este mesmo esquema pode ser comparado com o esquema totalmente homomórfico de Gentry, que se trata de um esquema totalmente homomórfico de construção mais simples, contudo essa simplicidade vem ao custo de que sua chave pública possui um tamanho estimado de $\tilde{O}(\lambda^{10})$, o que de acordo com Coron et al, torna inviável a aplicação em sistemas práticos. O esquema totalmente homomórfico DGHV com chave pública reduzida diminui o tamanho da chave pública gerada para $\tilde{O}(\lambda^7)$ criptografando de maneira quadrática os elementos da chave pública, ao invés de criptografá-los de maneira linear. Para fazê-lo foi utilizada a linguagem de programação Python, contando com a biblioteca de matemática e teoria numérica GMPY2.

Abstract: In this paper we implemented the fully homomorphic scheme with shorter key (DGHV with shorter key) proposed by Jean-Sébastien Coron, Avradip Mandal, David Naccache and Mehdi Tibouchi, which was published in the conference CRYPTO 2011, this same scheme can be compared with the Gentry's fully homomorphic scheme, being it a fully homomorphic scheme with a simpler construction, however this simplicity comes at the cost of the public key having an estimated size of $\tilde{O}(\lambda^{10})$, which according to Coron et al, makes it impossible to use in practical systems. The DGHV scheme over integers with shorter public key decreases the size of the generated public key to $\tilde{O}(\lambda^7)$, encrypting the information of the public key in a quadratic manner, instead of encrypting them in a linear way. To do so, the python programming language was used, with the library of mathematics and number theory GMPY2.

1. Introdução

A criptografia moderna tem como base problemas matemáticos relacionados à teoria de números, exemplo deste é o algoritmo RSA que usa a fatoração de números inteiros em números primos para a geração de suas chaves assimétricas [Rivest et al., 1978], outro exemplo disso é o uso de logaritmos discretos na criptografia de curvas elípticas [Miller, 1985], entretanto, o surgimento da teoria dos computadores quântico e do algoritmo de Shor tornou este tipo de solução vulnerável a ataques quânticos [Shor, 1994]. Para proteger dados, até mesmo da criptoanálise quântica, tivemos o surgimento de uma nova área dentro da segurança da informação, chamada de Criptografia Pós-Quântica. Essa área estuda a criação de algoritmos criptográficos resistentes a ataques

quânticos, tendo como base outros problemas matemático-computacionais tais como as classes de algoritmos baseados em *hash*, os baseados em reticulados, os baseados em código, entre outros. O tempo de execução de ataque para todos esses algoritmos é exponencial, mesmo para um computador quântico [Bernstein et al. 2008].

Dentre os esquemas criptográficos pós-quânticos, destacam-se aqueles que possuem a característica de serem totalmente homomórficos. Um esquema é dito totalmente homomórfico quando se é possível avaliar circuitos lógicos de profundidade arbitrária nos dados cifrados, de forma pública [Gentry, 2009]. Em outras palavras, existe uma primitiva *Eval*, que dada uma função $f()$ e um texto cifrado $E(m)$, sendo $E()$ a função criptográfica e “ m ” a mensagem, retorna $E(f(m))$.

A figura I ilustra os principais tipos de esquemas totalmente homomórficos existentes. O primeiro esquema totalmente homomórfico conhecido foi o proposto por Gentry em sua tese original, proposta em 2009, o mesmo incluía pela primeira vez a primitiva *recrypt*, que dava ao esquema, originalmente parcialmente homomórfico, a característica de ser totalmente homomórfico, que utiliza como base reticulados ideais. Estudando a tese de Gentry, e seu esquema de *bootstrapping* como base, Halevi, em parceria com o próprio Gentry, implementaram em 2010, o esquema totalmente homomórfico Gentry-Halevi, este basicamente o esquema original de Gentry, com inclusão de algumas melhorias.

Ainda em 2010, Dijk, Gentry, Halevi e Vaikuntanathan (DGHV), propuseram um esquema totalmente homomórfico que utiliza apenas álgebra modular sobre o anel de números inteiros. Este mesmo esquema foi posteriormente analisado por Coron em 2011, que propôs duas técnicas de redução de chave pública para o esquema, dando origem a duas variantes do esquema DGHV. A primeira variante de Coron, chamada de DGHV com chave reduzida, conta com a adição de novos parâmetros quadráticos adicionados as primitivas do esquema, armazenando apenas um pequeno conjunto da chave pública para então gerar a chave pública completa em tempo de execução. A segunda variante de Coron, chamada de DGHV com compactação de chave pública com troca de módulo, utiliza geradores de números pseudoaleatórios e armazena apenas as sementes e os fatores de correção para os números pseudoaleatórios, recuperando os valores em tempo de execução.

Em 2011, Brakerski e Vaikuntanathan, propuseram um novo tipo de esquema totalmente homomórfico, este é baseado no problema de aprendizagem com erros (sigla em inglês: LWE), este em específico aplicando a problemas LWE em anel (da sigla em inglês: RLWE), este aplica os resultados conhecidos para o RLWE diretamente no esquema. Já em 2012, Brakerski, Gentry e Vaikuntanathan, apresentaram um novo esquema totalmente homomórfico com base no problema LWE, este esquema funciona sem a necessidade de operação de *bootstrapping*, porém mantendo-a no esquema como forma de melhorar a eficiência.

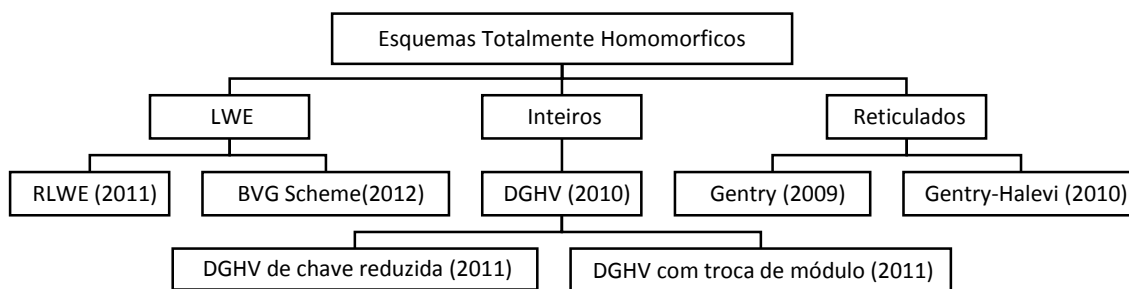


Figura I - Principais esquemas FHE

Baseado no trabalho de Gentry, dois tipos de esquemas homomórficos são conhecidos: O esquema de Gentry com reticulados ideais [Gentry, 2009] e o esquema DGHV sobre inteiros. O uso de reticulados na implementação não se mostrou uma das melhores alternativas, de acordo com Coron et al [Coron et al., 2011] e de Gentry [Dijk et al., 2010], pois o tamanho das chaves públicas geradas era na ordem de $\tilde{O}(\lambda^{10})$, com a implementação de Coron [Coron et al., 2011], na qual foram utilizados números inteiros na geração de chaves, obteve-se uma redução para um tamanho estimado de $\tilde{O}(\lambda^7)$ da chave pública. O presente trabalho tem como proposta implementar o esquema totalmente homomórfico sobre inteiros de chave reduzida proposto por Coron, [Coron et al., 2011].

2. Fundamentação teórica

O esquema implementado neste artigo é uma variante proposta por Coron do esquema originalmente proposto por Dijk, Gentry, Halevi e Vaikuntanathan (DGHV) no Eurocrypt 2010 [Dijk et al., 2010].

2.1. DGHV sobre inteiros

Segue uma breve descrição do sistema original sobre números inteiros, utilizado por Coron como base para seu trabalho. Gentry define que DGHV sobre inteiros utiliza como base um conjunto de inteiros públicos, $x_i = p \cdot q_i + r_i$, $0 \leq i \leq \tau$, onde o inteiro p é secreto [Gentry e Halevi, 2011], sendo dado um parâmetro de segurança λ , os seguintes parâmetros devem ser utilizados para compor o esquema SHE (do inglês, encriptação parcialmente homomórfica), que gera o FHE (do inglês, encriptação totalmente homomórfica) sobre inteiros:

- γ é o comprimento em bits de x_i 's;
- η é o comprimento em bits da chave secreta p ;
- ρ é o comprimento em bits do ruído r_i ;
- τ é o número de x_i 's na chave pública;
- ρ' é um parâmetro de ruído secundário utilizado para cifrar.

Que devem seguir as seguintes restrições, de forma a mitigar possíveis ataques:

- $\rho = \omega(\log \lambda)$, para proteção contra ataques de força bruta direcionados ao ruído;
- $\eta \geq \rho \cdot \Theta(\lambda \log^2 \lambda)$ para que seja possível realizar operações homomórficas para avaliar o “circuito de deciptação reduzido”;
- $\gamma = \omega(\eta^2 \cdot \log \lambda)$ para frustrar ataques baseados em retículos com aproximação pelo problema de MDC;
- $\tau \geq \gamma + \omega(\log \lambda)$ para reduzir a aproximação por MDC;
- $\rho' = \rho + \omega(\log \lambda)$ para o parâmetro de ruído secundário.

Dados esses parâmetros é possível gerar as primitivas do esquema DGHV, sendo que para um inteiro ímpar p de η -bit, seja utilizada uma distribuição sobre inteiros de γ -bit:

$$\mathcal{D}_{\gamma, \rho}(p) = \{ \text{Escolha } q \leftarrow \mathbb{Z} \cap \left[0, \frac{2^\gamma}{p}\right), r \leftarrow \mathbb{Z} \cap (-2^\rho, 2^\rho) : \text{Saída } x = q \cdot p + r \}$$

2.1.1. Primitivas Do DGHV

As primitivas deste esquema são quatro, *KeyGen*, *Encrypt*, *Decrypt* e *Evaluate*. *KeyGen* é a primitiva responsável pela geração do par de chaves do esquema, *Encrypt* responsável por gerar o texto cifrado, *Decrypt* responsável por decifrar o texto cifrado, e *Evaluate*, que executa, de forma pública, um circuito lógico sobre uma tupla de bits cifrados, e, que retorna o equivalente cifrado desse circuito aplicado aos dados originais. Sendo assim obtemos as primitivas como sendo:

i. *KeyGen*(λ):

Sendo a chave privada um inteiro ímpar de η -bits, $p \leftarrow (2\mathbb{Z} + 1) \cap [2^{\eta-1}, 2^\eta)$, para gerar a chave pública amostramos $x_i \leftarrow \mathcal{D}_{\gamma, \rho}(p)$ para $i = 0, \dots, \tau$, reordenando de forma que x_0 seja maior o maior elemento do conjunto, e recomeçando o processo a não ser que x_0 seja um número ímpar e $[x_0]_p$ seja par. Assim a chave pública é definida como $pk = \langle x_0, x_1, \dots, x_\tau \rangle$.

ii. *Encrypt*($pk, m \in \{0,1\}$):

Escolhe-se um subconjunto $S \subseteq \{1, 2, \dots, \tau\}$ aleatório, e um inteiro aleatório r entre $(-2^{\rho'}, 2^{\rho'})$, e gera o texto cifrado c como:

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

iii. *Evaluate*($pk, C, c_1, \dots, c_t$):

Dado o circuito C com t bits de entrada e t textos cifrados c_i , aplicar a adição e multiplicação das portas lógicas de C nos textos cifrados, executando todas as operações sobre os inteiros, e retornar o inteiro resultante.

iv. *Decrypt*(sk, c):

Tem como saída $m \leftarrow (c \bmod p) \bmod 2$. Note que $c \bmod p = c - p \cdot \lfloor c/p \rfloor$ e p é ímpar, então a função *decrypt* pode ser calculada como: $m \leftarrow [c]_2 \oplus [\lfloor c/p \rfloor]_2$.

Assim é descrito o esquema criptográfico parcialmente homomórfico, sendo este semanticamente seguro contra ataques de aproximação de MDC.

2.2. DGHV Sobre Inteiros Com Chave Pública Reduzida

Coron [Coron et.al., 2011] utilizou uma variante do esquema DGHV, onde foi adicionado um novo parâmetro β , sendo manipulados números inteiros $x'_{i,j}$, na forma de $x'_{i,j} = x_{i,0} \cdot x_{j,1} \bmod x_0$ para $1 \leq i, j \leq \beta$. Dessa forma, apenas 2β inteiros precisam ser armazenados para gerar os $\tau = \beta^2$ inteiros utilizados para a criptografia. Em outras palavras, o sistema executa a criptografia utilizando elementos na forma quadrática da chave pública, ao contrario da forma linear adotada anteriormente. Tal permite reduzir o tamanho da chave pública de τ para $2\sqrt{\tau}$ inteiros de γ bits [Coron et al, 2011].

Além disso, a fim de tornar esse esquema totalmente homomórfico, são necessários λ^3 elementos y com comprimento de $\kappa = \gamma + 2 + \lceil \log_2(\lambda + 1) \rceil$, o que aumentaria o tamanho da chave pública de $\tilde{O}(\lambda^7)$ para $\tilde{O}(\lambda^8)$. Coron então propôs que apenas o primeiro dos elementos de y fosse armazenado na chave, e, os outros gerados a partir de uma função geradora de números aleatórios. Dessa forma, o tamanho da chave é mantido na ordem de λ^7 , e, os elementos de y são recuperados em tempo de execução [Coron et al, 2011].

É apresentada a seguir a descrição completa do esquema proposto por Coron:

i. *KeyGen*(1^λ):

Gera um número ímpar p com η bits. Escolhe um número inteiro $q_0 \in [0, 2^\gamma / p)$, escolhido como um produto de dois primos aleatórios de λ^2 bits, e $x_0 = q_0 \cdot p$. Gera β pares inteiros de $x_{i,0}, x_{i,1}$ dentro do intervalo de $1 \leq i \leq \beta$:

$$x_{i,b} = p \cdot q_{i,b} + r_{i,b}, 1 \leq i \leq \beta, 0 \leq b \leq 1$$

onde, $r_{i,b}$ são inteiros entre $(-2^\rho, 2^\rho)$ e $q_{i,b}$ são inteiros aleatórios de ordem $[0, q_0)$. Sendo assim $pk^* = (x_0, x_{1,0}, x_{1,1}, \dots, x_{\beta,0}, x_{\beta,1})$.

Também gera os vetores $s^{(0)}$ e $s^{(1)}$, de comprimento $\lceil \sqrt{\theta} \rceil$, que seguem a condição de que $s_1^{(0)} = s_1^{(1)} = 1$, para cada um dos $\kappa \in [0, \sqrt{\theta})$ e $b = 0, 1$, onde há pelo menos um bit não zero entre os $s_i^{(b)}$ s, $k \lceil \sqrt{B} \rceil < i \leq (k+1) \lceil \sqrt{B} \rceil$, com $B = \theta/\theta$, e S sendo $S = \{(i,j): s_i^{(0)} \cdot s_j^{(1)} = 1\}$, contendo exatamente θ elementos.

Inicializa um gerador f de números pseudoaleatórios válido para todo o sistema com uma semente aleatória se , usando assim $f(se)$ para gerar $u_{i,j} \in [0, 2^{\kappa+1})$ para $1 \leq i, j \leq \lceil \sqrt{\theta} \rceil, (i,j) \neq (1,1)$. Então, atribuindo $u_{1,1}$ de forma que:

$$\sum_{(i,j) \in S} u_{i,j} = x_p \text{ mod } 2^{\kappa+1}$$

onde $x_p \leftarrow \lfloor 2^\kappa / p \rfloor$.

Assim, computando a cifra de $\sigma^{(b)}$ dos vetores $s^{(b)}$, escolhendo para cada $i \in [1, \lceil \sqrt{\theta} \rceil]$ e $b = 0, 1$, inteiros aleatórios $r'_{i,b} \in (-2^\rho, 2^\rho)$ e $q'_{i,b} \in [0, q_0)$, é determinado que:

$$\sigma_i^{(b)} = s_i^{(b)} + 2r'_{i,b} + p \cdot q'_{i,b} \text{ mod } x_0$$

Deste modo, temos como saída da primitiva KeyGen(), a chave privada sendo $sk = (s^{(0)}, s^{(1)})$, e a chave pública como $pk = (pk^*, se, u_{1,1}, \sigma^{(0)}, \sigma^{(1)})$.

ii. *Encrypt*($pk, m \in \{0,1\}$):

Escolha uma matriz de números aleatórios $b = (b_{i,j})_{1 \leq i,j \leq \beta} \in [0, 2^\alpha)^{\beta \times \beta}$ e um inteiro aleatório r no intervalo $(-2^{\rho'}, 2^{\rho'})$. Retorne o texto cifrado como:

$$c^* = m + 2r + 2 \sum_{1 \leq i,j \leq \beta} b_{ij} \cdot x_{i,0} \cdot x_{j,1} \text{ mod } x_0$$

iii. *Add*(pk, c_1^*, c_2^*): Retorna $c_1^* + c_2^* \text{ mod } x_0$

iv. *Mult*(pk, c_1^*, c_2^*): Retorna $c_1^* \cdot c_2^* \text{ mod } x_0$

v. *Expand*(pk, c^*):

Esse procedimento, a expansão do texto cifrado, recebe um texto cifrado c^* e computa a matriz associada z . Podemos pensar nesse procedimento como separado tanto do *Encrypt* quanto de *Decrypt*, já que pode ser executado de forma pública utilizando apenas o texto cifrado e dados públicos. Para tal, para cada $1 < i, j < \sqrt{\theta}$, primeiramente compute o número inteiro aleatório $u_{i,j}$ usando o gerador de números pseudoaleatórios $f(se)$, então calcule $y_{i,j} = u_{i,j} / 2^\kappa$ e então compute $z_{i,j}$:

$$z_{i,j} = \lfloor c^* \cdot y_{i,j} \rfloor_2$$

mantendo apenas $n = \lceil \log_2(\theta + 1) \rceil$ bits de precisão após o ponto binário. Defina a matriz $z = (z_{i,j})$. Retorne o texto cifrado expandido $c = (c^*, z)$

vi. *Decrypt*(sk, c^*, z): Retorna $m \leftarrow \left[c^* - \left[\sum_{i,j} s_i^{(0)} \cdot s_j^{(1)} \cdot z_{ij} \right] \right]_2$.

vii. *Recrypt*(pk, c^*, z):

Aplice o circuito de deciptação ao texto cifrado expandido Z e às chaves secretas encriptadas $\sigma_i^{(b)}$. Retorne o resultado como o texto cifrado renovado c_{new}^* .

Assim finalizamos a descrição completa do esquema DGHV Totalmente Homomórfico de chave pública reduzida, a primitiva *Evaluate*, foi fragmentada, dando origem a outras duas primitivas de nome *Add* e *Mult*, que emulam, respectivamente, o comportamento das portas lógicas *XOR* e *AND* aplicadas ao texto puro, de forma bit-a-bit. Tal esquema é seguro contra ataques baseados em MDC, como foi provado na sessão 4 da publicação que descreve essa variante do esquema [Coron et al, 2011].

3. Implementação

Para a implementação desse esquema, foi utilizada a linguagem Python 3.3, com a biblioteca GMPY2 para cálculos numéricos.

Inicialmente, foi criado um módulo, *theKey.py*, que é responsável pelo processo de escrita e leitura das classes das chaves:

```
1  import pickle
2  class pk():
3      '''classe para armazenar a chave privada'''
4      def __init__(self, pkAsk, se, u11, sigma0, sigma1, P):
5          self.pkAsk=pkAsk
6          self.se=se
7          self.u11=u11
8          self.sigma0=sigma0
9          self.sigma1=sigma1
10         self.P=P
11     class sk():
12         '''classe para armazenar a chave pública'''
13         def __init__(self, sz, su, P):
14             self.s0=sz
15             self.s1=su
16             self.P=P
17         def write(obj, name):
18             with open(name, 'wb') as arq:
19                 pickle.dump(obj, arq)
20         def read(name):
21             with open(name, 'rb') as arq:
22                 return pickle.load(arq)
```

As classes *pk* e *sk* funcionam como contêineres para os parâmetros da chave pública e da chave secreta, respectivamente, assim como dos parâmetros concretos. As funções *write()* e *read()* encapsulam o módulo *pickle* do Python, que executa a escrita e leitura de objetos do Python em arquivos binários.

Também foi criada uma classe para encapsular os parâmetros da criptografia, conforme descritos experimentalmente pro Gentry:

Para executar a geração do par de chaves criptográficas, é utilizada a seguinte função:

```
1  def keygen(file, size='toy'):
2      P.setPar(size)
3      tempo=-time.time()
```

```

4
5     p = genZi(P._eta)
6     q0, x0 = genX0(p, P._gamma, P._lambda)
7     listaX = genX(P._beta, P._rho, p, q0)
8     pkAsk = listaX
9     pkAsk.insert(0, x0)
10    while True:
11        s0,s1=genSk(P._theta, P._thetam)
12        if (s0.count(1)*s1.count(1)==15): break
14    se=int(time.time()*1000) #seed para RNG
15    _kappa=P._gamma+6
16    u11=genU11(se, s0, s1, P._theta, _kappa, p)
17    sigma0 = encryptVector(s0, p, q0, x0, P._rho)
18    sigma1 = encryptVector(s1, p, q0, x0, P._rho)
19    tempo+=time.time()
20    #pickle files
21    public=fheKey.pk(pkAsk, se, u11, sigma0, sigma1, P)
22    secret=fheKey.sk(s0, s1, P)
23    fheKey.write(public, 'pk_pickle_'+file)
24    fheKey.write(secret, 'sk_pickle_'+file)

```

A função *keygen* recebe como parâmetros o nome básico para os arquivos onde serão escritos a chave, e, opcionalmente o parâmetro de segurança que será utilizado. Caso esse parâmetro seja omitido, a função executará no padrão menos seguro.

Primeiramente a função gera um número inteiro ímpar P de η bits, os valores q_0 e x_0 , assim como uma lista de valores aleatórios x_i . Em seguida, a função gera a chave pública parcial pk^* (linhas 5-13). Em seguida, são gerados os vetores de bits aleatórios, e verificado se o peso de Hamming de $s^1 \times s^0 = \theta$ (linha 14).

A função então utiliza o tempo local, em milissegundos, como semente para o RNG (Gerador de números pseudoaleatórios). Utilizando o RNG do Python, é gerada uma matriz de valores U a partir dos vetores s^b . Com essa matriz é calculado o elemento u_{11} (linha 20). Por ultimo, são calculados os vetores σ^b , que são encriptações de s^b . Por fim, a função cria objetos da classe *fheKey.pk* e *fheKey.sk*, e os escreve no disco. Também é gerado um arquivo em texto puro dos valores computados que é utilizado para verificação.

Função de Encrypt:

```

1    def encrypt(pk, m):
2        if m != 0 and m != 1:
3            print('not a valid m')
4            return 0
5
6        alpha = pk.P._lambda
7        matrix = [[0 for i in range(pk.P._beta)] for j in
8        range(pk.P._beta)]
9        for i in range(pk.P._beta):
10           for j in range(pk.P._beta):
11               matrix[i][j]=random.randint(0, 2**alpha -
12           1)

```

```

11     pprime=2**pk.P._rho
12     r=random.randint(-pprime, pprime)
13     pkAsk=(pk.pkAsk).copy()
14     x0=pkAsk.pop(0)
15     xi0=pkAsk[::2]
16     xj1=pkAsk[1::2]
17     ## iniciando processo de encrypt
18     somatorio=mpz(0)
19     for i in range(pk.P._beta):
20         for j in range(pk.P._beta):
22             x=gmpy2.mul(xj1[j], xi0[i])
23             somatorio += gmpy2.mul(matrix[i][j], x)
24     c=(mpz(m)+2*r+2*somatorio)%x0
25     return c

```

A função de *encrypt* recebe como parâmetros a chave pública, e o bit a ser criptografado. Primeiramente, é criada uma matriz de números aleatórios $\beta \times \beta$ e populada com valores no intervalo de $(0, 2^\alpha]$ (linha 8). Então, é definido um número inteiro aleatório no intervalo de $(-2^\rho, 2^\rho)$. Por ultimo a função calcula o texto cifrado $c = m + 2r + 2 \sum_{1 \leq i, j \leq \beta} b_{ij} \cdot x_{i,0} \cdot x_{j,1} \bmod x_0$. O valor de c é retornado como um inteiro de precisão múltipla.

Função de expansão:

```

1  def expand(pk, cAsk):
2      #cria uma matriz de sqrt(theta) elementos
3      l=int(math.sqrt(pk.P._theta))
4      y=[[0 for i in range(l)] for i in range(l)]
5      kappa=pk.P._gamma+6
6      n=2 ##ceil(log2(thetaM+1))
7      gmpy2.get_context().precision=n
8      for i in range(l): #computa matriz y[i][j]
9          for j in range(l):
10             y[i][j]=float(gmpy2.mpfr(randomMatrix(i,
11 j, kappa, pk.se, l)/(2**kappa)))
12             y[0][0]=float(gmpy2.mpfr(pk.u1/2**kappa))
13             gmpy2.get_context().precision=21000
14             expand = [[1 for i in range(l)] for i in range(l)]
15             for i in range(l):
16                 for j in range(l):
17                     expand[i][j]=float((cAsk * y[i][j])%2)
18             return expand

```

A função de expansão recebe como parâmetros a chave pública, e a cifra c , e, a partir deles, computa a matriz associada z . Essa função é separada das primitivas *encrypt* e *decrypt*, já que pode ser computada de forma pública a partir apenas de dados públicos. Para tal, é criada uma matriz de y $\lceil \sqrt{\theta} \rceil$ elementos, que é populada usando o RNG(se). Então, a matriz z é calculada como $z_{i,j} = [c^* \cdot y_{i,j}]_2$.

Função de decriptação:

```

1  def decrypt(sk, cAsk, z):
2      soma=0
3      l=int(math.sqrt(sk.P._theta))
4      for i in range(l):
5          for j in range(l):
6              soma+=(sk.s0[i]*sk.s1[j]*z[i][j])
7      soma=mpz(gmpy2.round_away(soma))
8      m=(cAsk-soma)%2
9      return m

```

A função de decriptação recebe como parâmetros a chave secreta, a mensagem cifrada e a matriz z . Então, a função calcula $soma = \left[\sum_{i,j} s_i^{(0)} \cdot s_j^{(1)} \cdot z_{i,j} \right]$ e retorna o bit decriptado como $[c^* - soma]_2$.

Funções de cálculo homomórfico:

```

1  def add(pk, c1, c2):
2      soma=gmpy2.add(c1,c2)
3      return soma%pk.pkAsk[0]
4  def mul(pk, c1, c2):
5      mul= gmpy2.mul(c1,c2)
6      return mul%pk.pkAsk[0]

```

Essas funções são o equivalente as portas lógicas *XOR* e *AND* para os bits do texto cifrado. *XOR* é definido como uma soma de dois textos cifrados módulo x_0 e *AND* é definido como uma multiplicação de dois textos cifrados módulo x_0 .

Tabela 1. Tempos de execução

Parâmetro	<i>KeyGen</i>	<i>Encrypt</i>	<i>Decrypt</i>	<i>Expand</i>
<i>Toy</i>	0.6 s	0.00236 s	0.0001 s	1.28103 s
<i>Small</i>	10 s	0.01294 s	0.0002 s	8.08505 s
<i>Medium</i>	1 min 1 s	*	*	*
<i>Large</i>	11 min 21s	*	*	*

(*estouro de memória)

Os tempos de execução obtidos (ver tabela 1) foram gerados em um único núcleo de um processador Core i5 m520 com 2.40Ghz de frequência, em um computador com sistema operacional de 64 bits e 4 GB de memória.

4. Conclusão

A escolha da implementação em linguagem Python trouxe algumas facilidades na geração de código. Por exemplo, operações com listas tornam algumas tarefas extremamente simples de se realizar: Na geração de pares s para a função de *keygen*, um dos primeiros passos é gerar uma lista de tamanho $\sqrt{\Theta}$, inicialmente com todos os elementos iguais a zero e o primeiro elemento igual a um, sendo que essa tarefa é trivialmente executada em Python.

A maior dificuldade encontrada durante esse projeto foi, inicialmente, conseguir informações sobre os esquemas homomórficos e, transformar e interpretar as ideias, apresentadas quase sempre como fórmulas e abstrações matemáticas em códigos de

computador. Também foi desafiador compilar o trabalho original de Gentry, já que exigem procedimentos diferentes e específicos para a correta compilação do código.

Junto das facilidades inerentes a linguagem, as operações matemáticas de alto desempenho e as primitivas necessárias para números inteiros de precisão múltipla foram providas pela biblioteca de teoria numérica GMPY2. Tal biblioteca, além de disponibilizar uma primitiva de números inteiros de múltipla precisão, necessário para os cálculos homomórficos, permite que os cálculos com esses números sejam notavelmente mais rápidos que as primitivas disponibilizadas pelo Python.

Além disso, a programação a partir da descrição matemática se mostrou uma excelente maneira de se compreender o conceito e a matemática por trás do homomorfismo, permitindo dessa forma adquirir um conhecimento mais profundo sobre o funcionamento do esquema homomórfico.

A próxima etapa desse trabalho será adicionar a função de *decrypt*, necessária para se executar circuitos de profundidade arbitrária no *ciphertext*. Posteriormente, esse código será modificado para ser executado de forma paralela em GPGPUS, utilizando-se de recursos tais como pyCuda, e NumbaPro.

Referências bibliográficas

RIVEST, R. L., SHAMIR, A., AND ADLEMAN, L. M. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, 1978.

MILLER, V. Uses of elliptic curves in cryptography. In *Advances in Cryptology, Crypto 85, Lecture Notes in Computer Science*, pages 417–426. Springer, 1985
SHOR, P. W. Algorithms for quantum computation: discrete logarithms and factoring. In Shor, P. W., editor, *35th Annual Symposium on Foundations of Computer Science* (Santa Fe, NM, 1994), p. 124–134. IEEE Comput. Soc. Press, 1994.

BERNSTEIN, D. J., BUCHMANN, J. A., DAHMEN E. Post-Quantum Cryptography, Chicago and Darmstadt, 2008.

GENTRY, C., A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University, 2009, Disponível em: <http://crypto.stanford.edu/craig>.

CORON, J.S., MANDAL, A., NACCACHE, D. e TIBOUCHI, M., Fully Homomorphic Encryption over the Integers with Shorter Public Keys. In P. Rogaway (Ed.), *CRYPTO 2011, LNCS, vol. 6841*, Springer, pp. 487–504. Versão complete disponível em IACR eprint, 2011.

DIJK., M. VAN, GENTRY, C., HALEVI, S. e VAIKUNTANATHAN, V., Fully homomorphic encryption over the integers. In H. Gilbert (Ed.), *EUROCRYPT 2010, LNCS, vol. 6110*, Springer, p. 24–43, 2010.

GENTRY, C., e HALEVI, S., "Implementing Gentry's fully-homomorphic encryption scheme," *Advances in Cryptology-EUROCRYPT 2011*, p. 129–148, 2011.

GENTRY, C., Fully Homomorphic Encryption Using Ideal Lattices, Symposium on Theory of Computing - STOC, p.169–178, 2009.